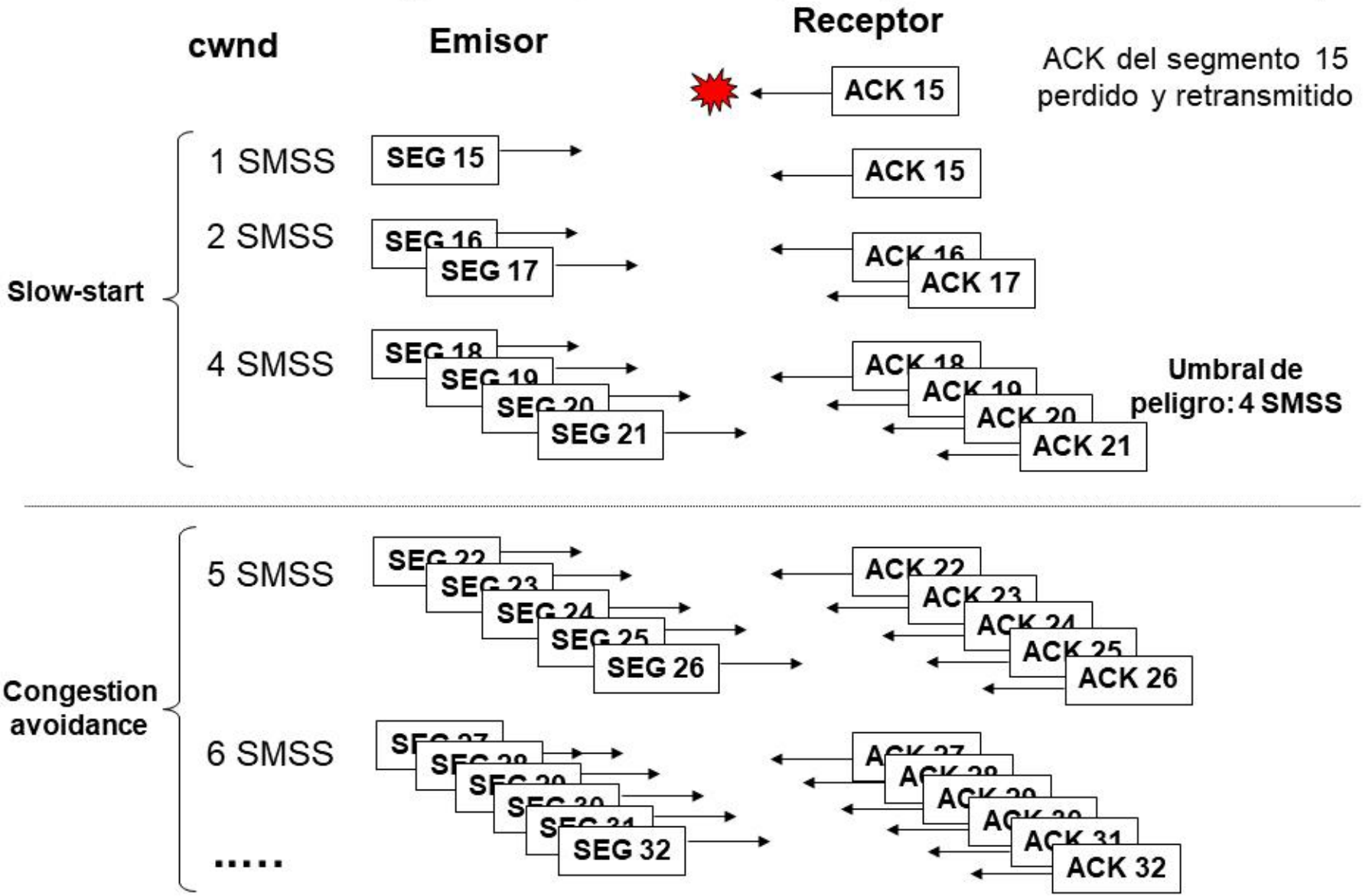
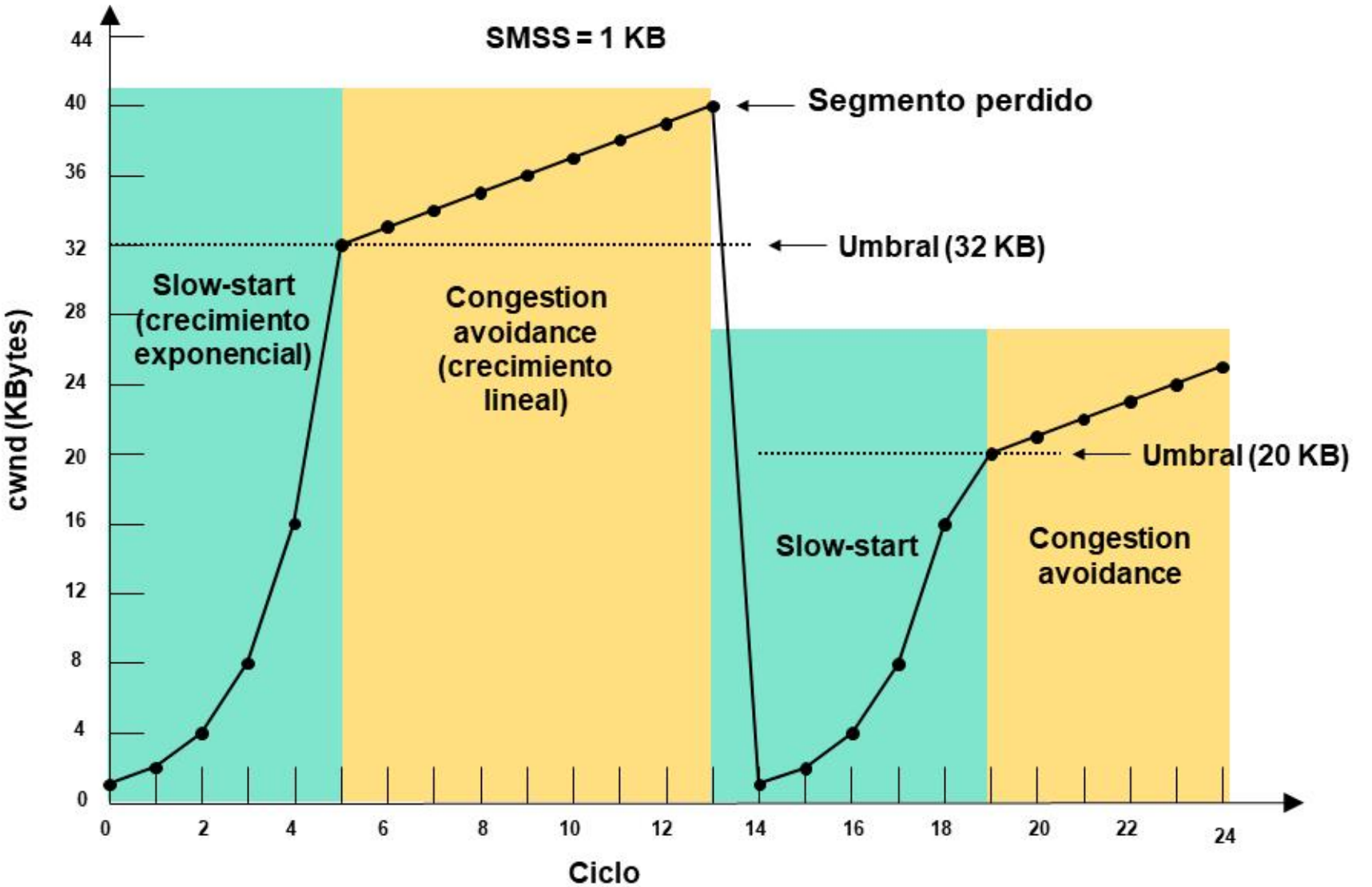
Evitación de la congestión (Congestion avoidance, fase 2)

- Cuando se pierde un segmento:
  1. La cwnd vuelve a su valor inicial (de uno a cuatro SMSS)
  2. Se fija un 'umbral de peligro' en un valor igual a la mitad de la cwnd que había cuando se produjo la pérdida.
  3. La cwnd crece exponencialmente como antes (slow-start) hasta el umbral de peligro; a partir de ahí crece linealmente, sólo un SMSS en cada ciclo

Control de congestión, fase 2 (congestion avoidance)



Algoritmo TCP-Tahoe (1988)



Ejercicio: control de congestión

- Una conexión TCP sobre un medio físico poco fiable pierde exactamente una trama de cada 10. El nivel de enlace descarta las tramas erróneas, sin pedir reenvío (CRC):
  - Detalle como se producirá el envío de los segmentos y como evolucionará la ventana de congestión en el TCP emisor (se supone que se ha negociado utilizar SACK)
  - Estime cualitativaente la merma de rendimiento que cabe esperar debido a la tasa de error:
    A. Menor del 10%
    B. Alrededor del 10%
    C. Mayor del 10%
  - Diga como influiría el RTT en el rendimiento

Ejercicio: Evolución ventana de congestión

	cwnd (KB)	Trama	Segmento	Umbral peligro (KB)
S.S.	1	1	1	64
	2	2,3	2,3	64
	4	4,5,6,7	4,5,6,7	64
	8	8,9,10,11,12,13,14,15	8,9,10,11,12,13,14,15	64
S.S.	1	16	10	4
	2	17,18	16,17	4
	4	19,20,21,22	18,19,20,21	4
S.S.	1	23	19	2
	2	24,25	22,23	2
C.A.	3	26,27,28	24,25,26	2
	4	29,30,31,32	27,28,29,30	2
S.S.	1	33	28	2
	2	34,35	31,32	2
C.A.	3	36,37,38	33,34,35	2
	4	39,40,41,42	36,37,38,39	2
S.S.	1	43	37	2
	2	44,45	40,41	2
C.A.	3	46,47,48	42,43,44	2
	4	49,50,51,52	45,46,47,48	2
S.S.	1	53	46	2
	2	54,55	49,50	2

Ejercicio: solución

- La merma de rendimiento será siempre superior al 10%, ya que además de tener que reenviar el 10% de los paquetes tenemos que estar continuamente reiniciando la ventana de congestión.
- Cuanto mayor sea el RTT mayor será la pérdida. Con un RTT cero la pérdida sería del 10%

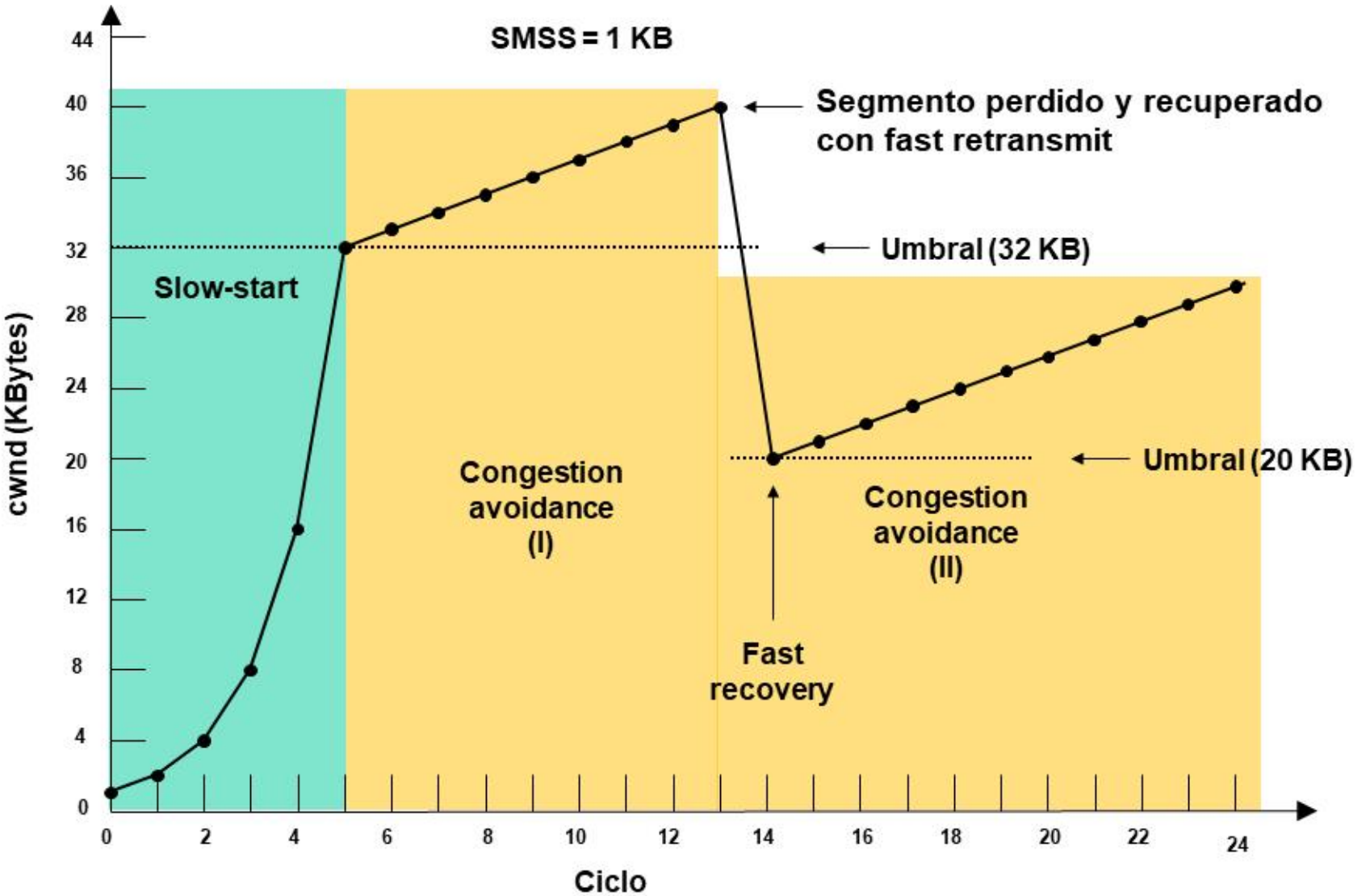
Fast recovery

- Van Jacobson introdujo una mejora en el algoritmo de congestión de TCP-Tahoe llamada fast recovery. Observó que la pérdida de un segmento podía detectarse de dos maneras:
  - Por agotamiento del timeout del ACK
  - Por la recepción de ACK duplicados (fast retransmit)
- En TCP Tahoe (1988) ambos casos se trataban del mismo modo: umbral de peligro, slow-start, congestion-avoidance
- Van Jacobson pensó que, en el caso del fast-retransmit, se podía aplicar un correctivo menos contundente, ya que después de todo el problema se había resuelto con el reenvío de un solo segmento.

Fast Recovery

- Van Jacobson decidió saltarse, en el caso del fast retransmit, la fase del slow-start, es decir, cuando se produjera un fast retransmit:
  1. TCP fijaría el umbral de peligro en la mitad de la cwnd vigente en ese momento
  2. Fijaría la nueva cwnd en el umbral de peligro
  3. Creceria linealmente a partir de ahí (congestion avoidance), pero sin empezar desde el principio
- Este mecanismo lo denominó 'fast recovery' y lo incorporó en 1990 en el 4.3BSD-Reno, por lo que se conoce como TCP-Reno

Algoritmo TCP-Reno (1990)



Algoritmos de congestión de TCP

- El control de congestión de TCP es objeto de continuos estudios. Todos los algoritmos actuales tienen como base a TCP-Reno. Los más utilizados actualmente son:
  - TCP New-Reno: FreeBSD, Linux < 2.6.8
  - TCP BIC (Binary Increase Congestion): Linux 2.6.8-18
  - TCP CUBIC: Linux ≥ 2.6.19
  - TCP PRR (Proportional Rate Reduction): Linux ≥3.2
  - Compound TCP: Windows Vista y sigtes.
- El algoritmo utilizado no ha de negociarse, puede ser diferente en cada lado.
- Ver en Wikipedia 'TCP Congestion Control'

Congestión y Errores de transmisión en TCP

- El mecanismo implícito de detección de congestión que utiliza TCP supone que las pérdidas de segmentos se deben a congestión.
- Si la pérdida de tramas se debe a errores será erróneamente interpretada como congestión por TCP
- Se iniciará entonces el slow-start y bajará el caudal de forma considerable, sobre todo en conexiones con RTT elevado. P. ej. un enlace de 1 Gb/s con un RTT de 10ms y una pérdida del 1% da un rendimiento útil de 2 Mb/s
- Para un buen rendimiento TCP necesita que el canal de comunicación no tenga errores; si los hay el nivel de enlace debe habilitar los mecanismos adecuados para corregirlos (códigos FEC) o retransmitir las tramas erróneas (WiFi)

Efecto de la pérdida de paquetes en el rendimiento de TCP para diversos valores de RTT (enlace de 1 Gbps)

